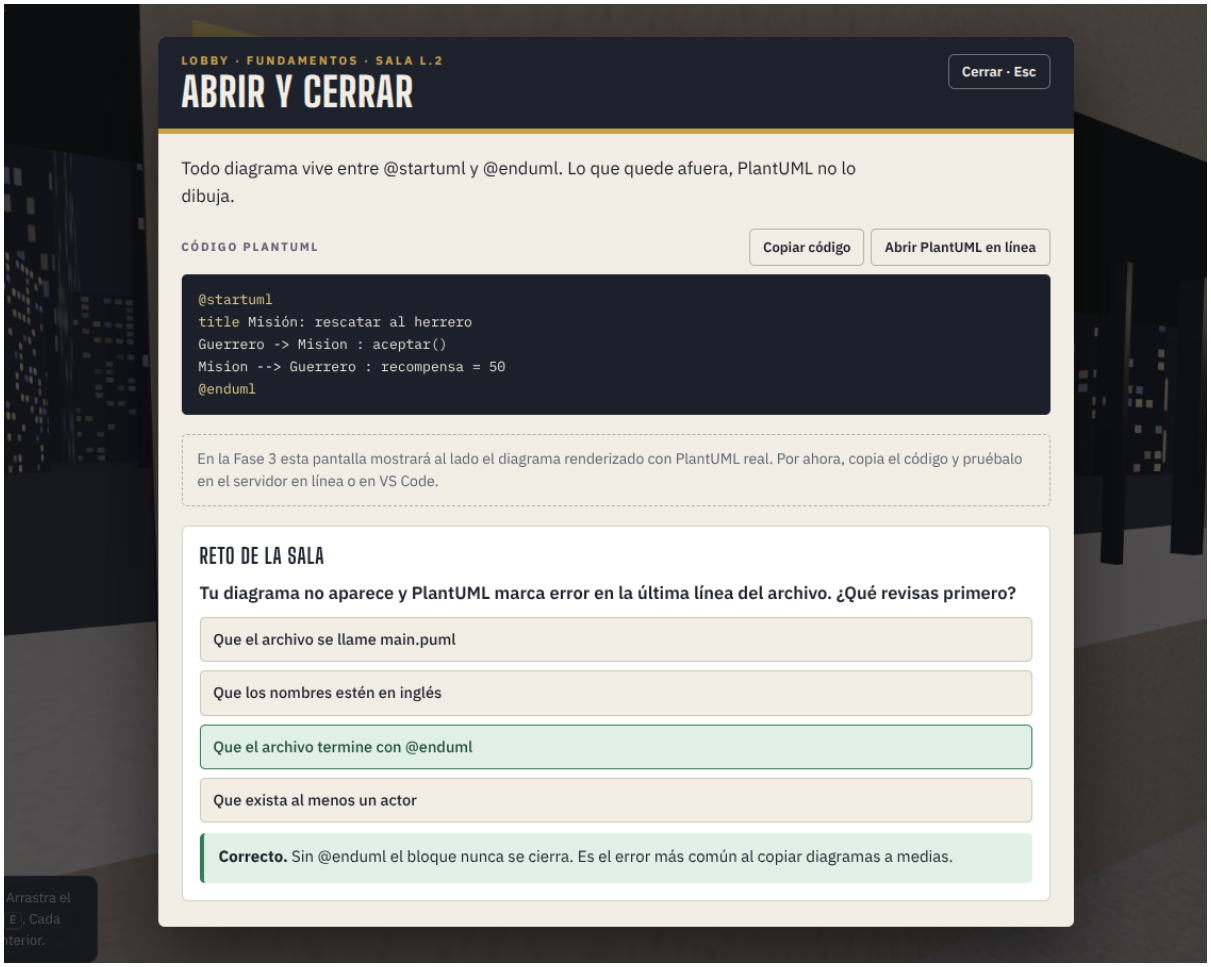
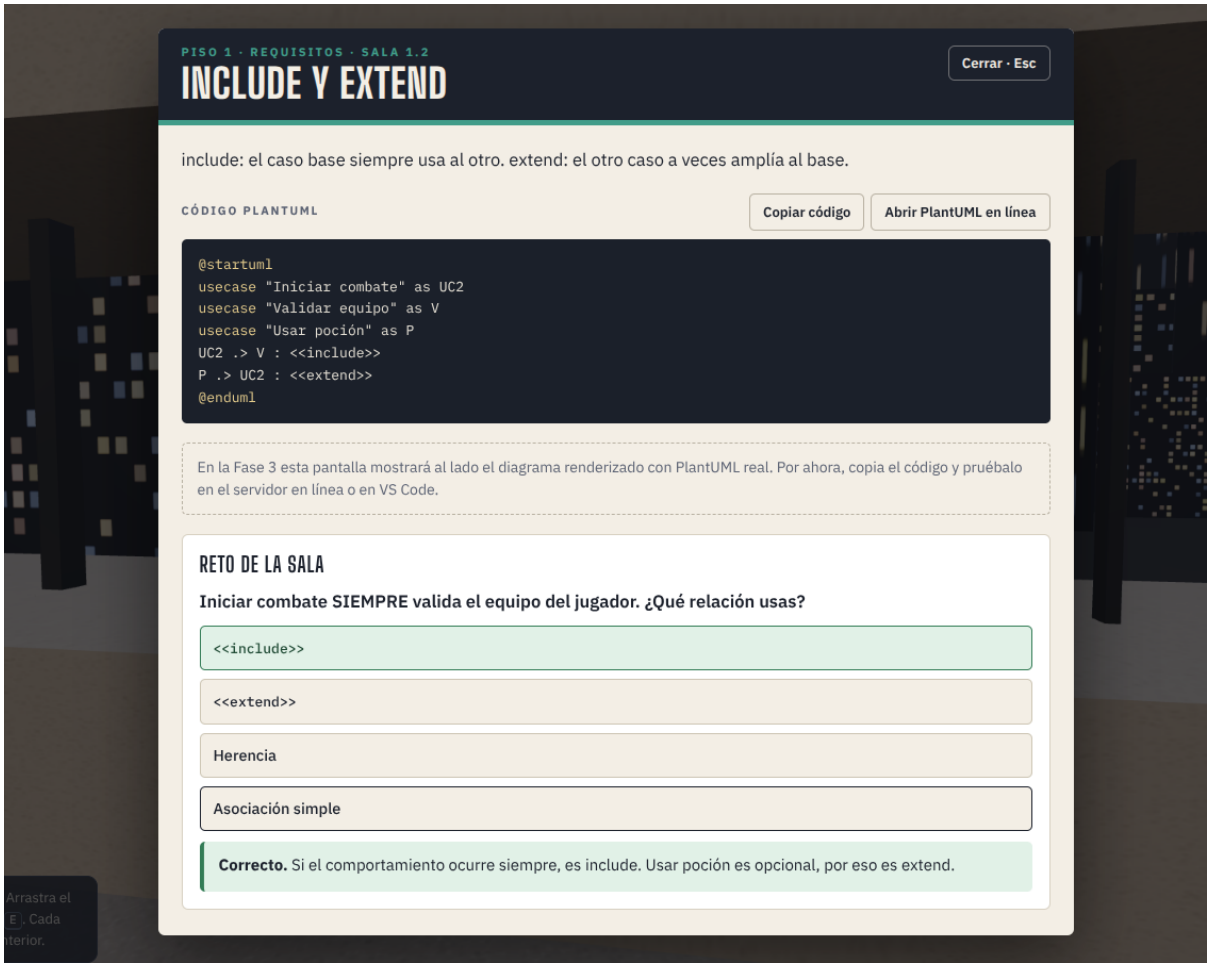

LOBBY · FUNDAMENTOS · SALA L.1
QUÉ ES PLANTUML
Cerrar · Esc
Escribes texto plano y PlantUML lo convierte en un diagrama. El archivo se versiona en GitHub como cualquier código.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
Jugador -> Personaje : atacar()
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué hace PlantUML con este archivo de texto?
Lo ejecuta como un programa Java
Lo convierte en un diagrama que puedes ver y exportar
Lo compila a un archivo .class
Lo guarda en una base de datos


LOBBY · FUNDAMENTOS · SALA L.4
TEMAS Y ESTILO
Cerrar · Esc
@startuml
!theme cerulean
skinparam classAttributeIconSize 0
class Guerrero {
- fuerza : int
}
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
Si cambias !theme cerulean por !theme plain, ¿qué cambia?
Las relaciones entre clases
Solo la apariencia del diagrama
El tipo de diagrama
Los atributos de Guerrero
Correcto. Los temas solo afectan la presentación. El modelo (clases, atributos, relaciones) queda igual.


LOBBY · FUNDAMENTOS · SALA L.3
COMENTARIOS Y NOTAS
Cerrar · Esc
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
' la vida nunca baja de 0
class Personaje
note right of Personaje
vida entre 0 y vidaMaxima
end note
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Cuál de estas líneas NO se ve en el diagrama dibujado?
' la vida nunca baja de 0
note right of Personaje
title Arena de combate
class Personaje
Correcto. La comilla simple marca un comentario. La nota, el título y la clase sí se dibujan.


LOBBY · FUNDAMENTOS · SALA L.2
ABRIR Y CERRAR
Cerrar · Esc
Todo diagrama vive entre @startuml y @enduml. Lo que quede afuera, PlantUML no lo dibuja.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
title Misión: rescatar al herrero
Guerrero -> Mision : aceptar()
Mision --> Guerrero : recompensa = 50
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
Tu diagrama no aparece y PlantUML marca error en la última línea del archivo. ¿Qué revisas primero?
Que el archivo se llame main.puml
Que los nombres estén en inglés
Que el archivo termine con @enduml
Que exista al menos un actor
Correcto. Sin @enduml el bloque nunca se cierra. Es el error más común al copiar diagramas a medias.

Piso 1


PISO 1 · REQUISITOS · SALA 1.2
INCLUDE Y EXTEND
Cerrar · Esc
include: el caso base siempre usa al otro. extend: el otro caso a veces amplía al base.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
usecase "Iniciar combate" as UC2
usecase "Validar equipo" as V
usecase "Usar poción" as P
UC2 .> V : <<include>>
P .> UC2 : <<extend>>
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
Iniciar combate SIEMPRE valida el equipo del jugador. ¿Qué relación usas?
<<include>>
<<extend>>
Herencia
Asociación simple
Correcto. Si el comportamiento ocurre siempre, es include. Usar poción es opcional, por eso es extend.


PISO 1 · REQUISITOS · SALA 1.1
CASOS DE USO
Cerrar · Esc
Muestran qué puede hacer cada actor con el sistema. El rectángulo marca el límite del sistema.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
left to right direction
actor Jugador
rectangle "RPG Manager" {
usecase "Crear personaje" as UC1
usecase "Iniciar combate" as UC2
}
Jugador --> UC1
Jugador --> UC2
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué representa rectangle "RPG Manager"?
Una clase del sistema
Un actor externo
El límite del sistema
La base de datos
Correcto. Los casos de uso van dentro del límite del sistema; los actores quedan afuera.



PISO 1 · REQUISITOS · SALA 1.3

# ACTIVIDADES: EL FLUJO

Cerrar · Esc

Describen un proceso paso a paso, con decisiones. Cada acción va entre dos puntos y punto y coma.

CÓDIGO PLANTUML

Copiar código | Abrir PlantUML en línea

```
@startuml
start
:Elegir personaje;
if (¿vida > 0?) then (sí)
  :Entrar a la Arena;
else (no)
  :Mostrar "Personaje caído";
endif
stop
@enduml
```

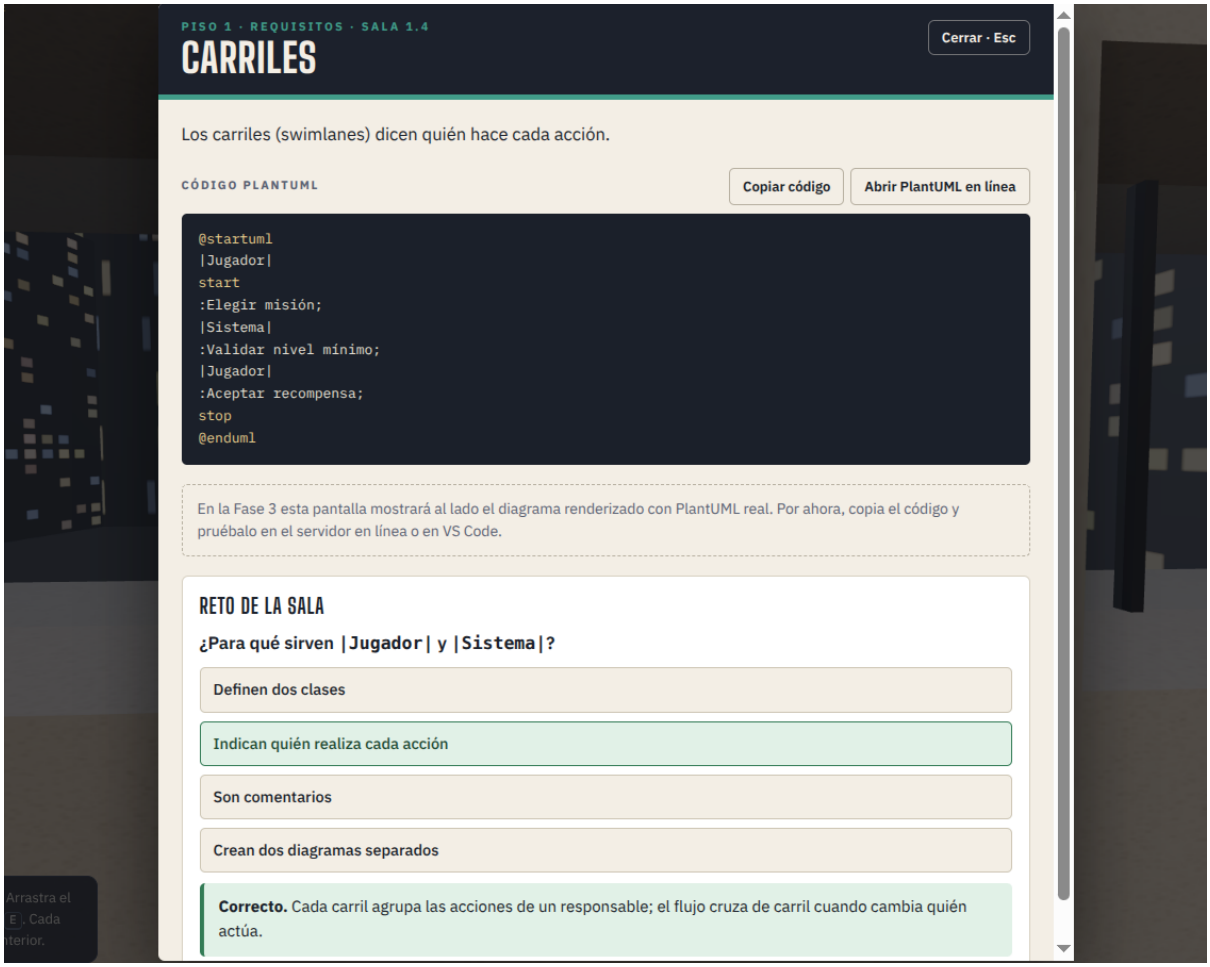
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.

## RETO DE LA SALA

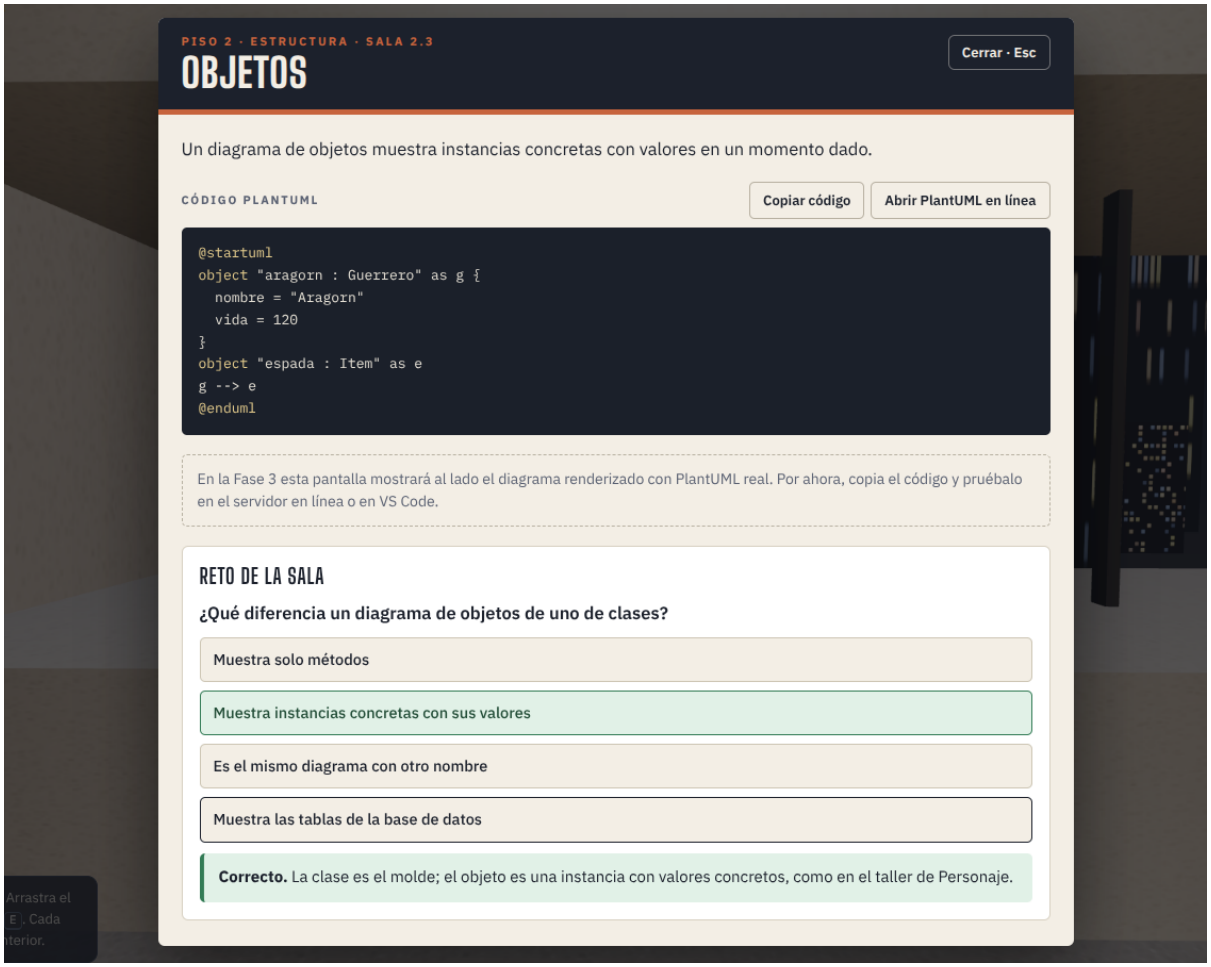
**¿Cómo se escribe una acción en un diagrama de actividades?**

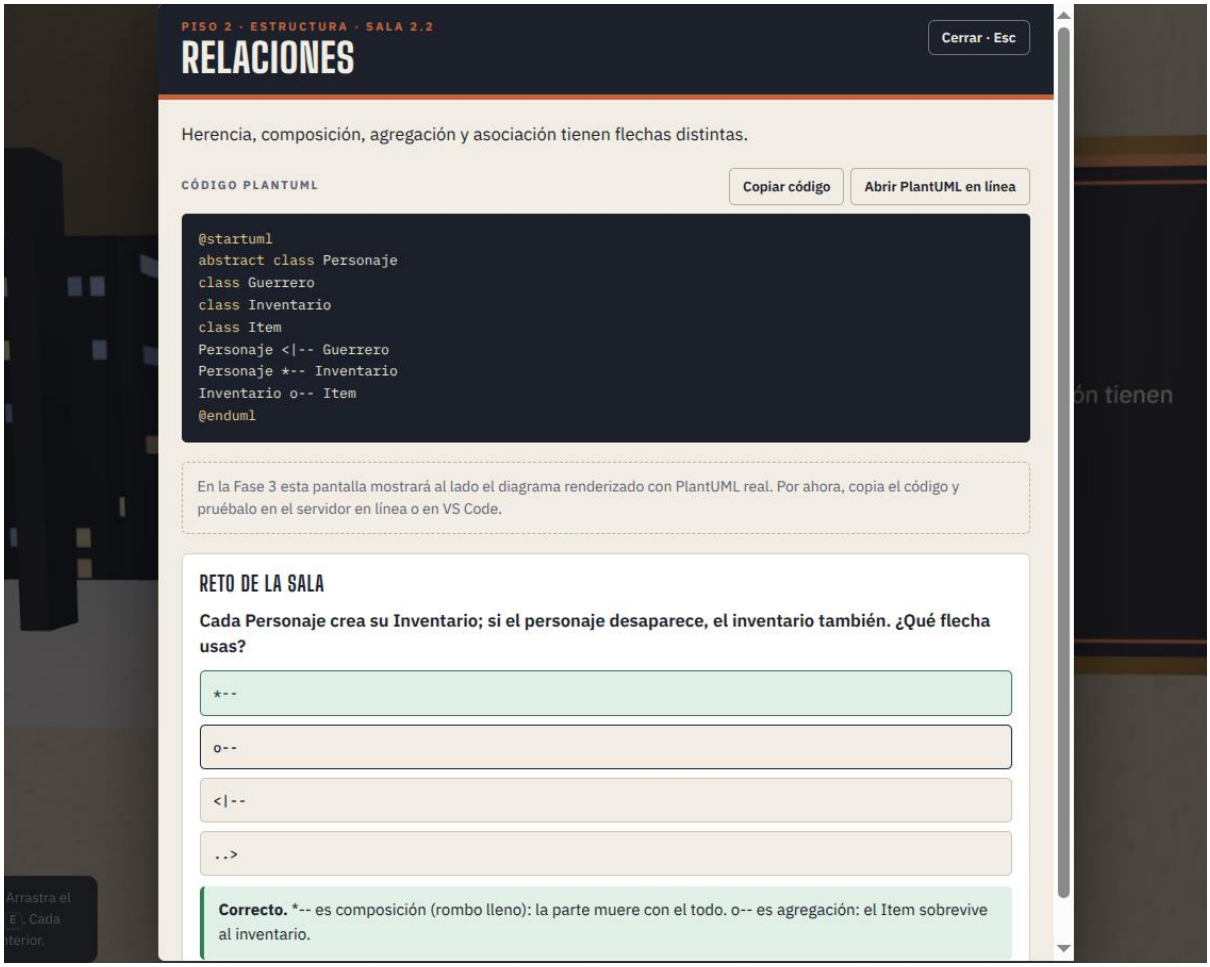
- `(Entrar a la Arena)`
- `[Entrar a la Arena]`
- `:Entrar a la Arena;`
- `"Entrar a la Arena"`

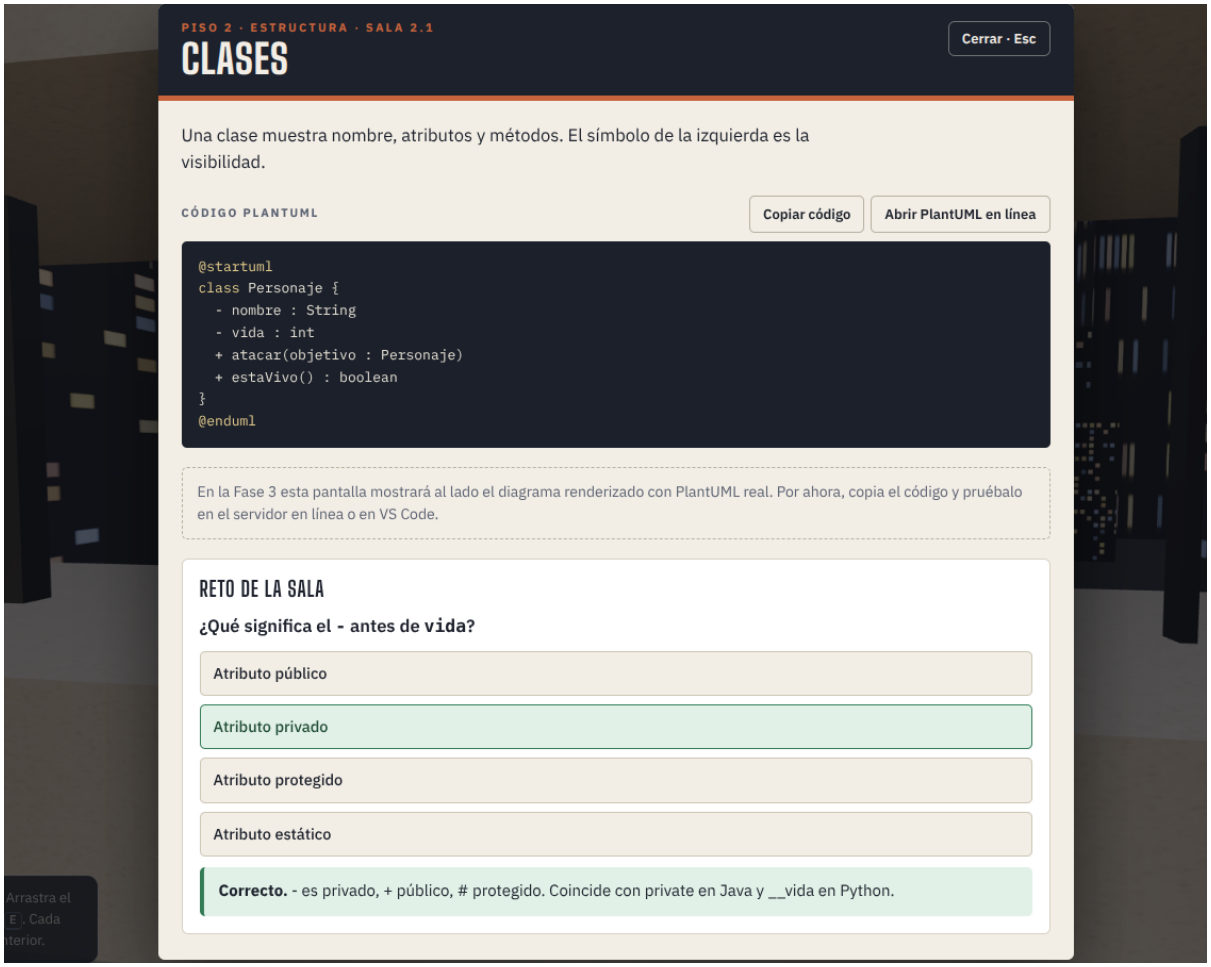
**Correcto.** La sintaxis es :texto; — dos puntos al inicio y punto y coma al final.


PISO 1 · REQUISITOS · SALA 1.4
CARRILES
Cerrar · Esc
Los carriles (swimlanes) dicen quién hace cada acción.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
|Jugador|
start
:Elegir misión;
|Sistema|
:Validar nivel mínimo;
|Jugador|
:Aceptar recompensa;
stop
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Para qué sirven |Jugador| y |Sistema|?
Definen dos clases
Indican quién realiza cada acción
Son comentarios
Crean dos diagramas separados
Correcto. Cada carril agrupa las acciones de un responsable; el flujo cruza de carril cuando cambia quién actúa.

Piso 2


PISO 2 · ESTRUCTURA · SALA 2.3
OBJETOS
Cerrar · Esc
Un diagrama de objetos muestra instancias concretas con valores en un momento dado.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
object "aragorn : Guerrero" as g {
nombre = "Aragorn"
vida = 120
}
object "espada : Item" as e
g --> e
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué diferencia un diagrama de objetos de uno de clases?
Muestra solo métodos
Muestra instancias concretas con sus valores
Es el mismo diagrama con otro nombre
Muestra las tablas de la base de datos
Correcto. La clase es el molde; el objeto es una instancia con valores concretos, como en el taller de Personaje.

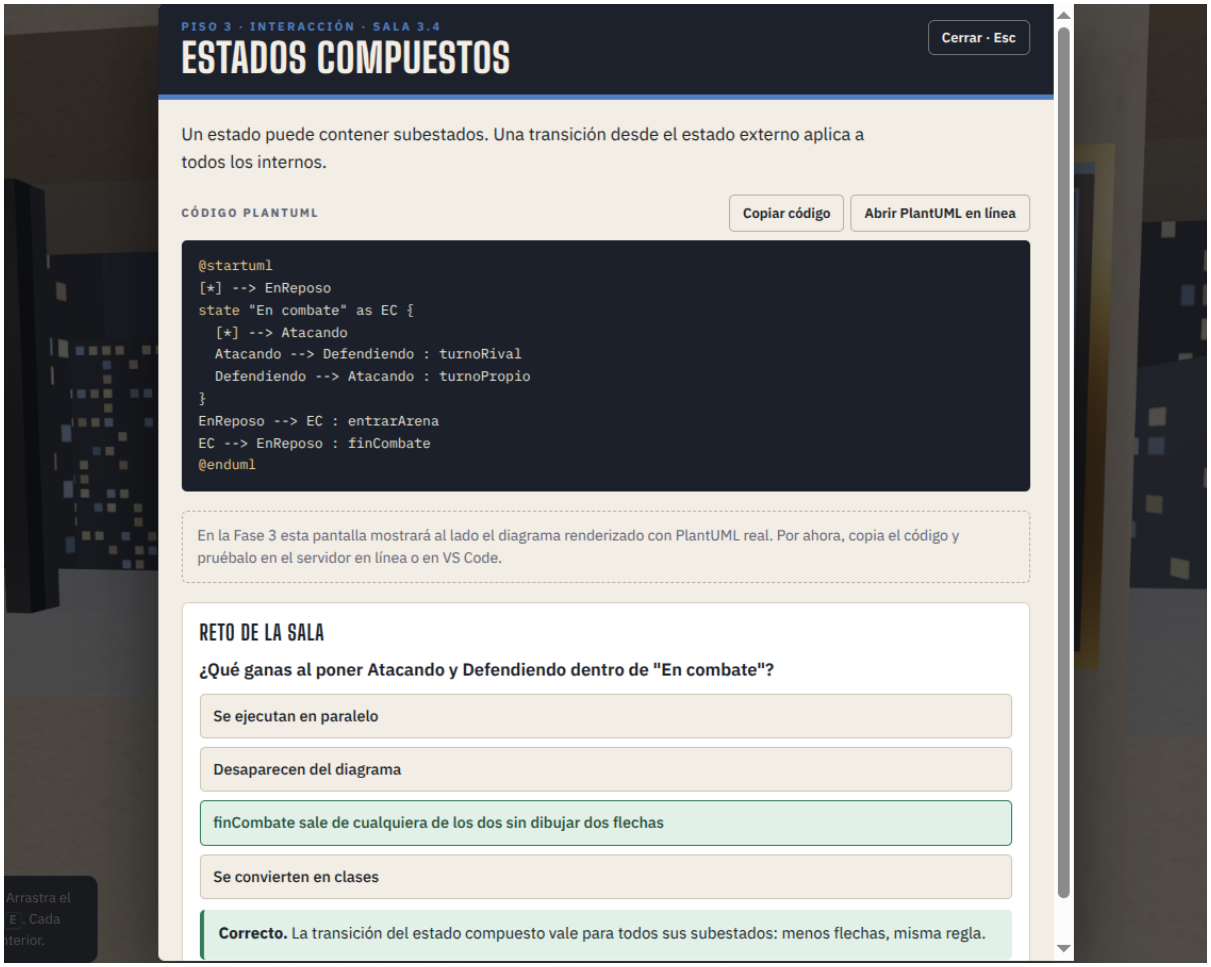

PISO 2 · ESTRUCTURA · SALA 2.2
RELACIONES
Cerrar · Esc
Herencia, composición, agregación y asociación tienen flechas distintas.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
abstract class Personaje
class Guerrero
class Inventario
class Item
Personaje <|-- Guerrero
Personaje *-- Inventario
Inventario o-- Item
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
Cada Personaje crea su Inventario; si el personaje desaparece, el inventario también. ¿Qué flecha usas?
*--
o--
<|--
..>
Correcto. *-- es composición (rombo lleno): la parte muere con el todo. o-- es agregación: el Item sobrevive al inventario.

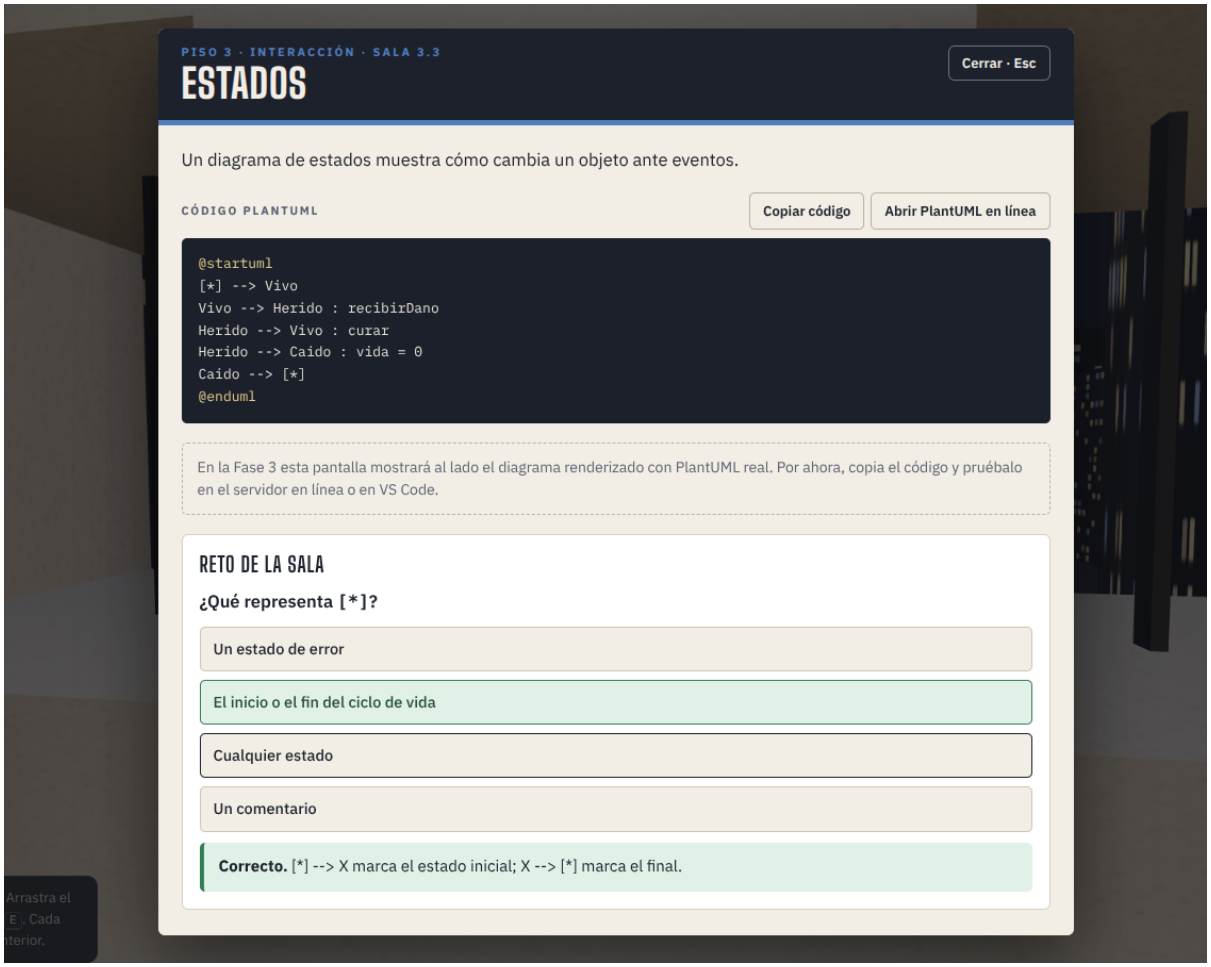

PISO 2 · ESTRUCTURA · SALA 2.1
CLASES
Cerrar · Esc
Una clase muestra nombre, atributos y métodos. El símbolo de la izquierda es la visibilidad.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
class Personaje {
- nombre : String
- vida : int
+ atacar(objetivo : Personaje)
+ estaVivo() : boolean
}
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué significa el - antes de vida?
Atributo público
Atributo privado
Atributo protegido
Atributo estático
Correcto. - es privado, + público, # protegido. Coincide con private en Java y __vida en Python.


PISO 2 · ESTRUCTURA · SALA 2.4
PAQUETES
Cerrar · Esc
Los paquetes agrupan clases y muestran dependencias entre módulos.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
package modelo {
class Personaje
class Item
}
package combate {
class ArenaCombate
}
combate ..> modelo
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué expresa combate ..> modelo?
combate hereda de modelo
modelo contiene a combate
combate depende de modelo (lo usa)
Son el mismo paquete
Correcto. ..> es una dependencia: ArenaCombate usa Personaje, por eso combate depende de modelo.

piso 3

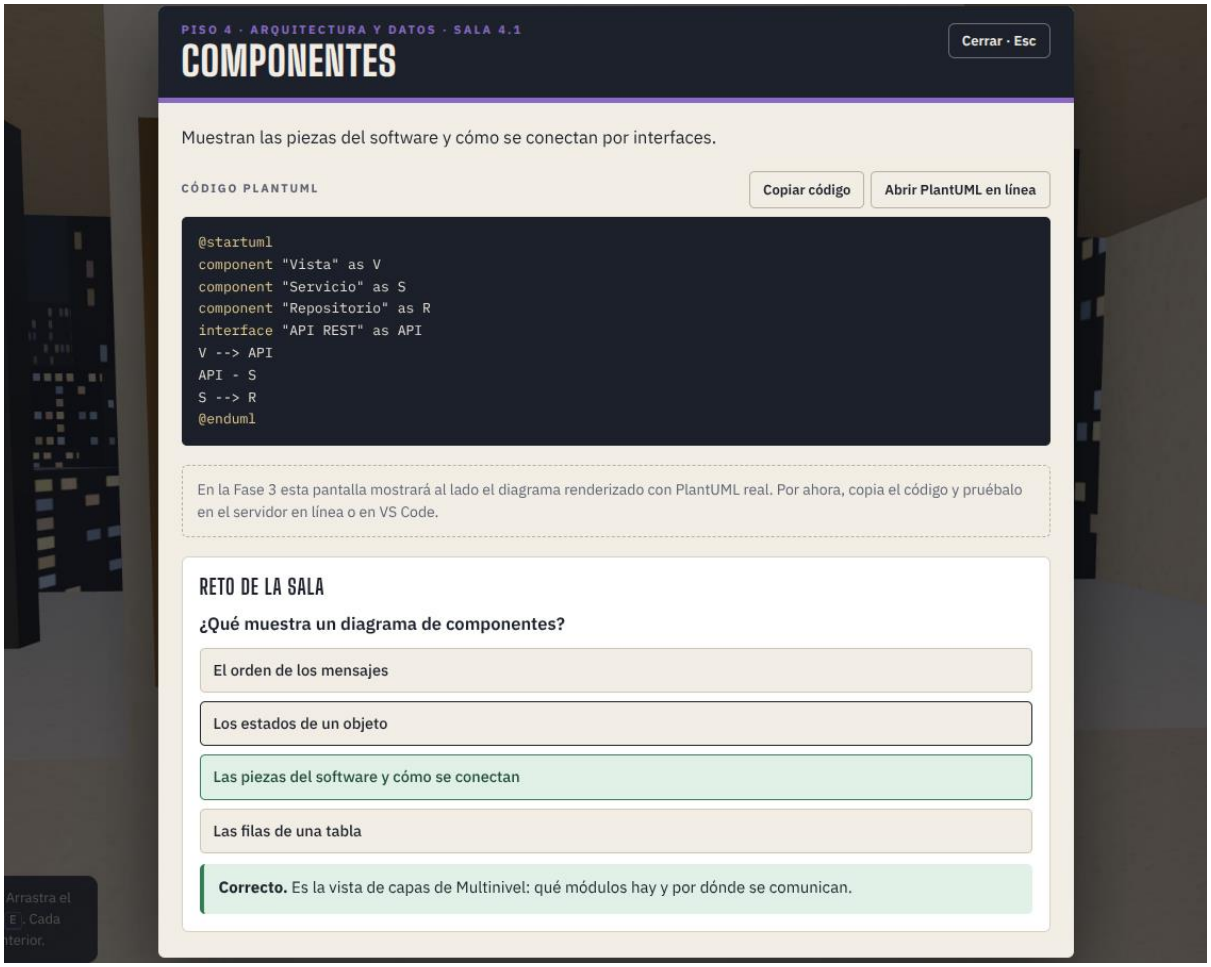

PISO 3 · INTERACCIÓN · SALA 3.1
SECUENCIA
Cerrar · Esc
Muestra mensajes entre participantes en orden de tiempo, de arriba hacia abajo.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
actor Jugador
participant Arena
participant Guerrero
Jugador -> Arena : iniciarCombate()
Arena -> Guerrero : atacar(monstruo)
Guerrero --> Arena : dano = 30
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué indica la flecha punteada - ->?
Herencia
Un retorno o respuesta
Que el mensaje falló
Un comentario
Correcto. -> es una llamada; --> es la respuesta que vuelve al que llamó.

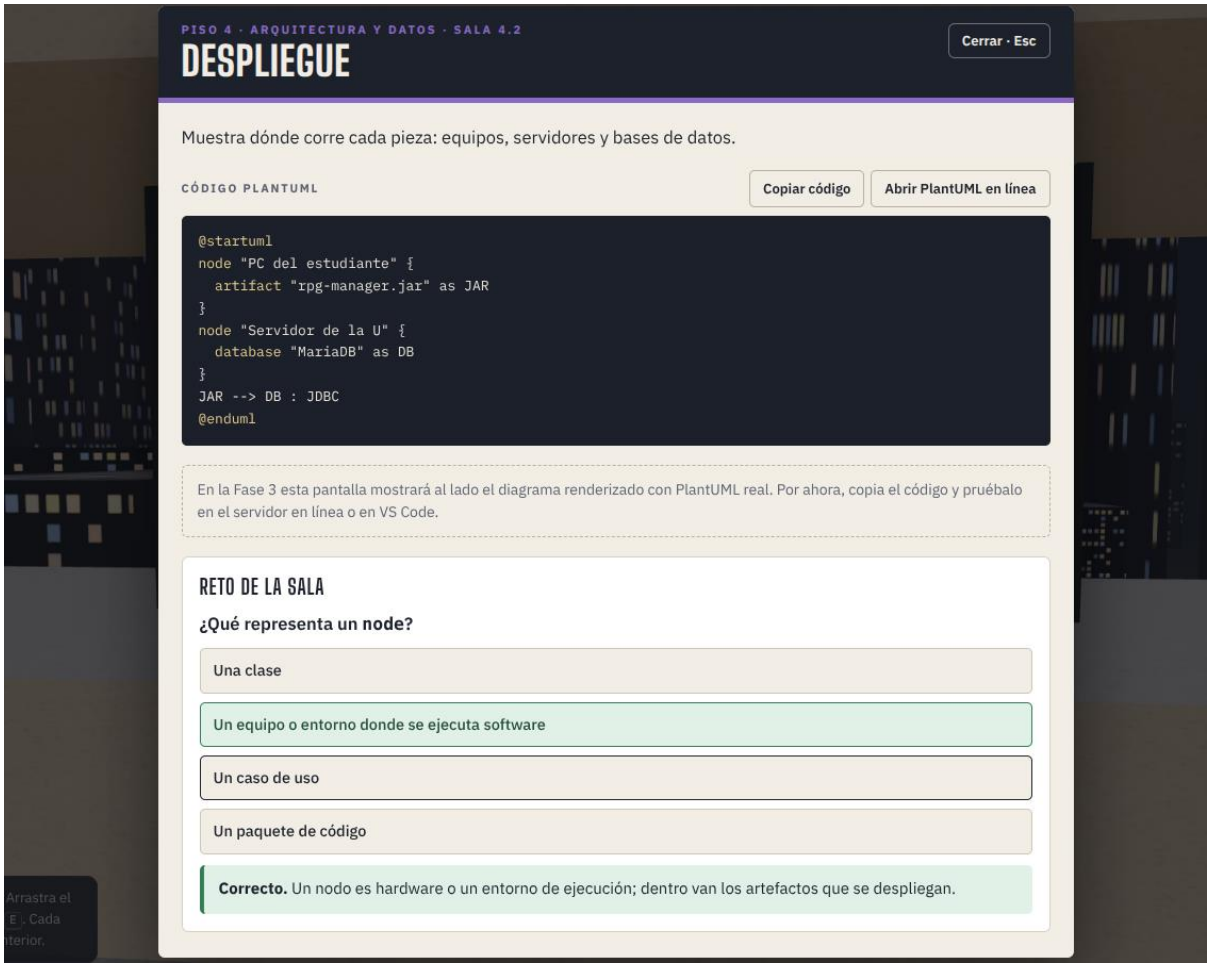

PISO 3 · INTERACCIÓN · SALA 3.4
ESTADOS COMPUESTOS
Cerrar · Esc
Un estado puede contener subestados. Una transición desde el estado externo aplica a todos los internos.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
[*] --> EnReposo
state "En combate" as EC {
[*] --> Atacando
Atacando --> Defendiendo : turnoRival
Defendiendo --> Atacando : turnoPropio
}
EnReposo --> EC : entrarArena
EC --> EnReposo : finCombate
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué ganas al poner Atacando y Defendiendo dentro de "En combate"?
Se ejecutan en paralelo
Desaparecen del diagrama
finCombate sale de cualquiera de los dos sin dibujar dos flechas
Se convierten en clases
Correcto. La transición del estado compuesto vale para todos sus subestados: menos flechas, misma regla.


PISO 3 · INTERACCIÓN · SALA 3.3
ESTADOS
Cerrar · Esc
Un diagrama de estados muestra cómo cambia un objeto ante eventos.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
[*] --> Vivo
Vivo --> Herido : recibirDano
Herido --> Vivo : curar
Herido --> Caido : vida = 0
Caido --> [*]
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué representa [*]?
Un estado de error
El inicio o el fin del ciclo de vida
Cualquier estado
Un comentario
Correcto. [*] --> X marca el estado inicial; X --> [*] marca el final.

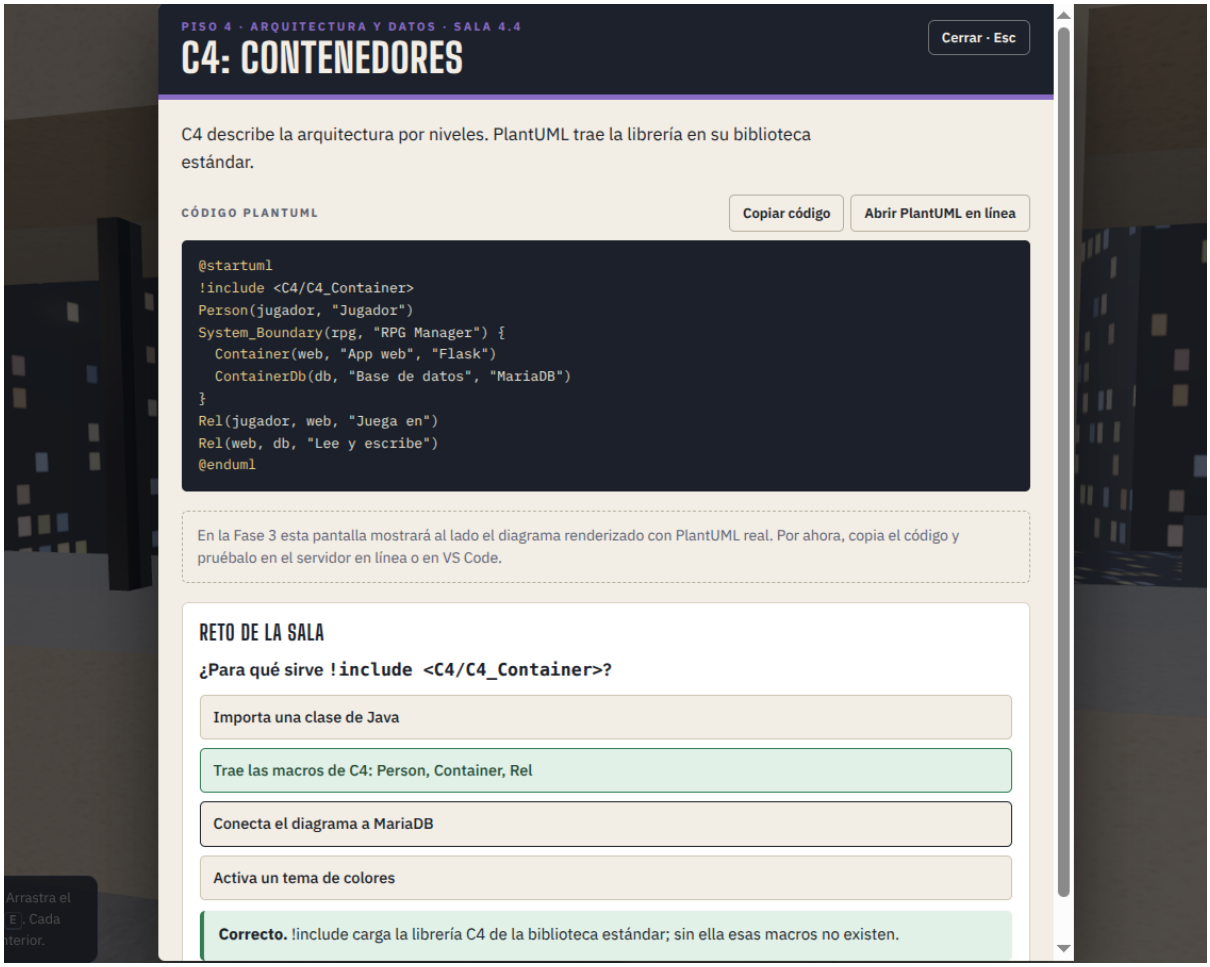

PISO 3 · INTERACCIÓN · SALA 3.2
FRAGMENTOS
Cerrar · Esc
loop repite, alt elige entre caminos, opt ocurre solo si se cumple algo.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
loop mientras ambos sigan vivos
Arena -> Guerrero : atacar()
alt mana >= 20
Arena -> Mago : lanzarHechizo()
else sin mana
Arena -> Mago : ataqueBasico()
end
end
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
La Arena repite rondas hasta que uno de los dos caiga. ¿Qué fragmento usas?
alt
opt
loop
ref
Correcto. loop representa repetición, como el ciclo while de la Arena de combate.

Piso 4

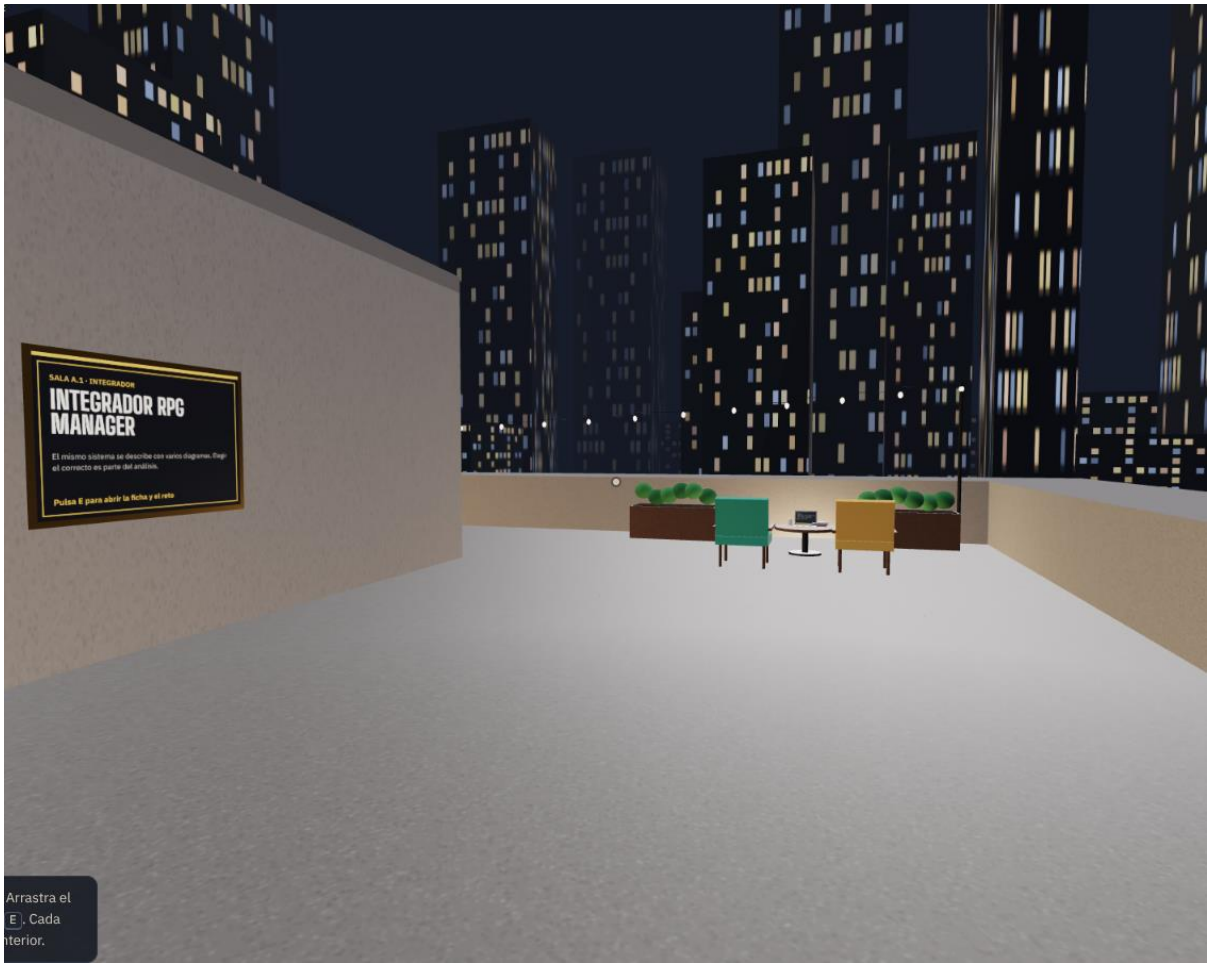

PISO 4 · ARQUITECTURA Y DATOS · SALA 4.1
COMPONENTES
Cerrar · Esc
Muestran las piezas del software y cómo se conectan por interfaces.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
component "Vista" as V
component "Servicio" as S
component "Repositorio" as R
interface "API REST" as API
V --> API
API - S
S --> R
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué muestra un diagrama de componentes?
El orden de los mensajes
Los estados de un objeto
Las piezas del software y cómo se conectan
Las filas de una tabla
Correcto. Es la vista de capas de Multinivel: qué módulos hay y por dónde se comunican.

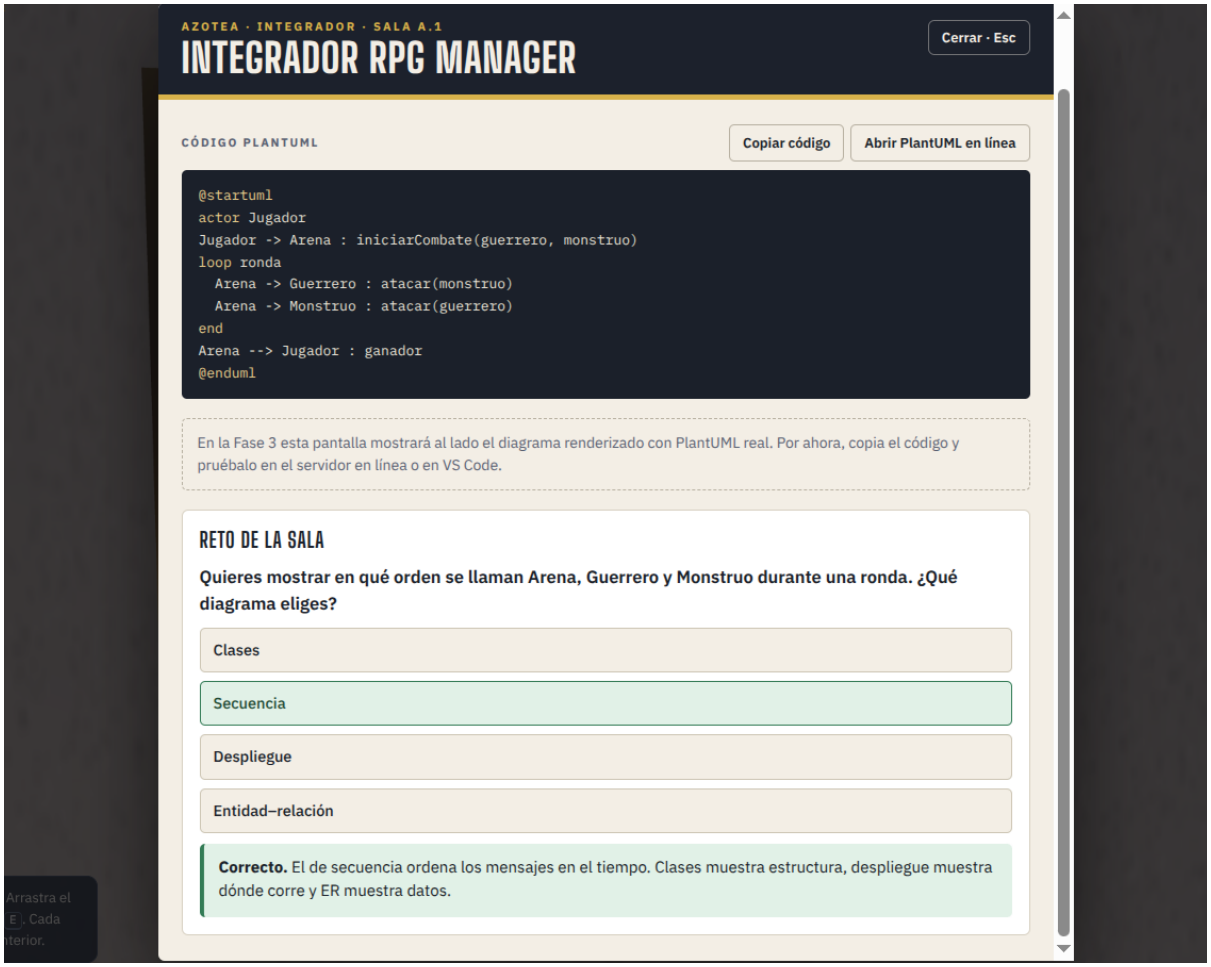

PISO 4 · ARQUITECTURA Y DATOS · SALA 4.2
DESPLIEGUE
Cerrar · Esc
Muestra dónde corre cada pieza: equipos, servidores y bases de datos.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
node "PC del estudiante" {
artifact "rpg-manager.jar" as JAR
}
node "Servidor de la U" {
database "MariaDB" as DB
}
JAR --> DB : JDBC
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Qué representa un node?
Una clase
Un equipo o entorno donde se ejecuta software
Un caso de uso
Un paquete de código
Correcto. Un nodo es hardware o un entorno de ejecución; dentro van los artefactos que se despliegan.

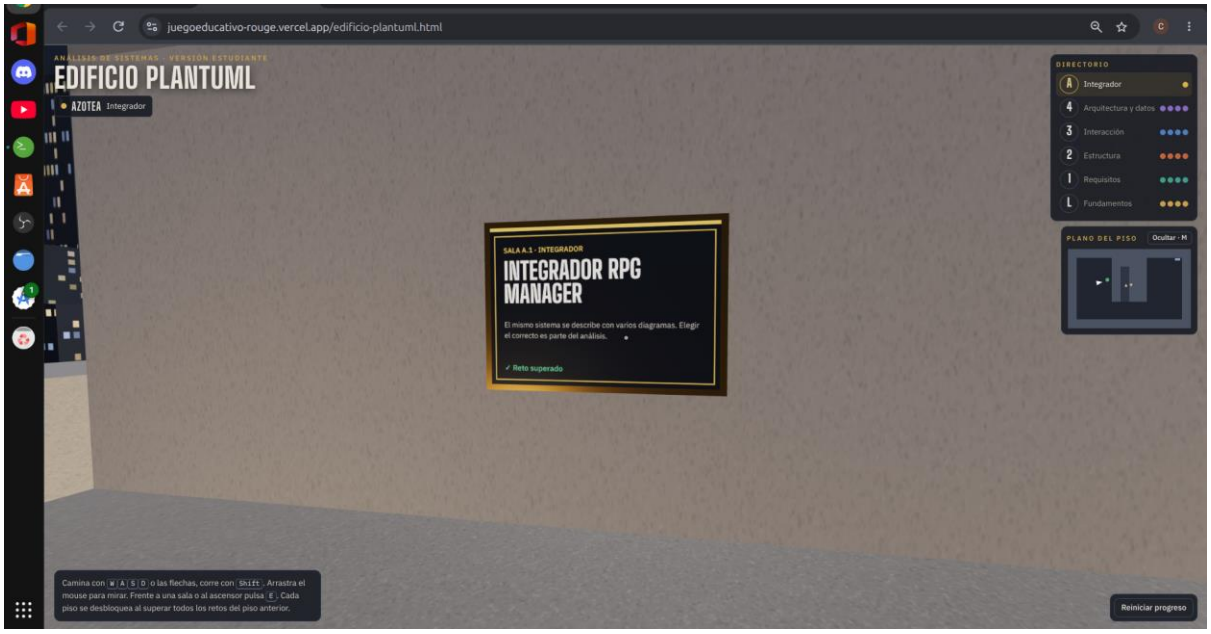

PISO 4 · ARQUITECTURA Y DATOS · SALA 4.3
ENTIDAD–RELACIÓN
Cerrar · Esc
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
entity Personaje {
* id : INT <<PK>>
--
nombre : VARCHAR(50)
}
entity Item {
* id : INT <<PK>>
--
personaje_id : INT <<FK>>
}
Personaje ||--o{ Item
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Cómo se lee Personaje ||--o{ Item?
Un personaje tiene exactamente un ítem
Un personaje tiene cero o muchos ítems
Muchos personajes comparten muchos ítems
Un ítem tiene muchos personajes
Correcto. Notación pata de gallo: cada Item pertenece a un Personaje; un Personaje puede tener 0..n ítems.


PISO 4 · ARQUITECTURA Y DATOS · SALA 4.4
C4: CONTENEDORES
Cerrar · Esc
C4 describe la arquitectura por niveles. PlantUML trae la librería en su biblioteca estándar.
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
!include <C4/C4_Container>
Person(jugador, "Jugador")
System_Boundary(rpg, "RPG Manager") {
Container(web, "App web", "Flask")
ContainerDb(db, "Base de datos", "MariaDB")
}
Rel(jugador, web, "Juega en")
Rel(web, db, "Lee y escribe")
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
¿Para qué sirve !include <C4/C4_Container>?
Importa una clase de Java
Trae las macros de C4: Person, Container, Rel
Conecta el diagrama a MariaDB
Activa un tema de colores
Correcto. !include carga la librería C4 de la biblioteca estándar; sin ella esas macros no existen.

azotea


SALA A.1 · INTEGRADOR
INTEGRADOR RPG MANAGER
El mismo sistema se describe con varios diagramas. Elige el correcto es parte del análisis.
Pulsa E para abrir la ficha y el reto
Arrastra el
E
. Cada
nterior.


AZOTEA · INTEGRADOR · SALA A.1
INTEGRADOR RPG MANAGER
Cerrar · Esc
CÓDIGO PLANTUML
Copiar código
Abrir PlantUML en línea
@startuml
actor Jugador
Jugador -> Arena : iniciarCombate(guerrero, monstruo)
loop ronda
Arena -> Guerrero : atacar(monstruo)
Arena -> Monstruo : atacar(guerrero)
end
Arena --> Jugador : ganador
@enduml
En la Fase 3 esta pantalla mostrará al lado el diagrama renderizado con PlantUML real. Por ahora, copia el código y pruébalo en el servidor en línea o en VS Code.
RETO DE LA SALA
Quieres mostrar en qué orden se llaman Arena, Guerrero y Monstruo durante una ronda. ¿Qué diagrama eliges?
Clases
Secuencia
Despliegue
Entidad–relación
Correcto. El de secuencia ordena los mensajes en el tiempo. Clases muestra estructura, despliegue muestra dónde corre y ER muestra datos.


juegoeducativo-rouge.vercel.app/edificio-plantuml.html
EDIFICIO PLANTUML
AZOTEA Integrador
INTEGRADOR RPG MANAGER
Reiniciar progreso